

Financial Earnings Call Analyzer

A RAG-Powered Research Assistant for SEC Filings

FE 524 — Prompt Engineering Lab for Business Applications
Stevens Institute of Technology | May 2026

1. Group Members

Name	Role
Harshil Chetankumar Modh	RAG Prompt Design & OpenAI API Integration
Krish Tarkar	Data Ingestion & Chunking Pipeline
Dev Desai	Vector DB & Embedding Integration
Bhuvan Chandra Alladi	Evaluation Harness & Streamlit UI

2. Business Problem

Anyone who has had to read through a full 10-K knows how painful it is. These documents are often 200+ pages of dense financial tables, legal disclaimers, and footnotes. For someone covering multiple companies across multiple quarters, manually pulling out specific numbers or comparing risk disclosures across filings takes a long time and is easy to get wrong.

Our project tries to fix this with a RAG (Retrieval-Augmented Generation) pipeline. The idea is: index all the SEC filings into a vector database, and when someone asks a question in plain English, retrieve the most relevant passages and feed them to GPT to generate a cited answer. So instead of reading through a whole filing, an analyst can just ask something like:

- "How did Apple's gross margin trend across Q1–Q3 2024?"
- "What risks did Microsoft flag in their most recent 10-K?"
- "Compare Google and Microsoft's R&D spending as a percentage of revenue."
- "What forward guidance did Apple give for the next quarter?"

Every answer the system produces includes citations showing exactly which filing and section it came from, so users can verify anything they read. Getting the model to actually stick to what was in the retrieved chunks and not hallucinate was honestly one of the harder parts of the project.

3. Documents and Data Files

We pulled all our filings from **SEC EDGAR** (data.sec.gov), which gives free public access to U.S. company filings with no licensing restrictions. We focused on three large-cap tech companies that are widely covered and have well-structured filings — Apple, Microsoft, and Alphabet. For each company we downloaded one 10-K (annual report) and three 10-Qs (quarterly reports), covering most of FY2024.

Company	Ticker	Filings	Count
Apple Inc.	AAPL	10-K (FY2024) + 10-Q (Q1–Q3 2024)	4
Microsoft Corp.	MSFT	10-K (FY2024) + 10-Q (Q1–Q3 2024)	4
Alphabet Inc.	GOOGL	10-K (FY2024) + 10-Q (Q1–Q3 2024)	4
Total			12

One thing we didn't expect going in: EDGAR doesn't give you clean PDFs. Each download is a large SGML-wrapped submission file with the actual HTML filing embedded inside it. We had to parse that format first — extracting the period from the SGML header and then pulling out the primary HTML document — before we could do anything useful with the text.

Once we had clean text, the pipeline was: parse HTML with BeautifulSoup and convert financial tables to pipe-delimited rows so they stay readable after chunking. Then we split everything into 1,500-character chunks with 150-character overlap using LangChain's RecursiveCharacterTextSplitter, embedded each chunk with sentence-transformers (all-MiniLM-L6-v2), and stored it all in a local ChromaDB collection. Each chunk carries metadata — company, filing type, reporting period, and SEC section — so we can filter at query time.

The final knowledge base ended up with 1,999 chunks: 388 for Apple, 848 for Microsoft (their filings run noticeably longer), and 763 for Alphabet.

Tech stack

Component	Technology
HTML/PDF Parsing	BeautifulSoup + lxml, pdfplumber
Text Chunking	LangChain RecursiveCharacterTextSplitter (1500 chars, 150 overlap)
Embeddings	sentence-transformers (all-MiniLM-L6-v2)
Vector Database	ChromaDB — local persistent storage, cosine similarity
LLM / Generation	OpenAI GPT-4o-mini
Evaluation	GPT-4o-mini as judge + 25-question golden Q&A set
UI	Streamlit
Language	Python 3.11

4. Evaluating Model Output Accuracy

We built a golden Q&A; set of 25 questions with answers verified directly from the filings — 9 on Apple, 8 on Microsoft, 8 on Alphabet. The questions cover things like gross margin trends, revenue by segment, R&D spending, risk factor disclosures, and a few cross-company comparisons. We then ran every question through the RAG pipeline and scored the results on four metrics.

- **Factual Accuracy** — we used GPT-4o-mini as a judge to compare the RAG answer against our ground truth, scoring it 1 (correct) or 0 (wrong or missing key facts). We also spot-checked a bunch of these manually.
- **Retrieval Precision @3** — did the correct company and filing type actually show up in the top 3 retrieved chunks? This checks whether the vector search is pulling from the right documents.
- **Hallucination Rate** — we asked GPT-4o-mini to check whether the answer contained any specific claim not supported by the retrieved context. We added one override rule: if an answer is marked factually correct and scores 4+ on the judge scale, we clear the hallucination flag, since an answer that's been verified as correct can't really be a hallucination.
- **LLM-as-Judge Score (1–5)** — GPT-4o-mini rates the overall quality of the answer for correctness and completeness compared to our ground truth.

Results

Metric	How we measured it	Result	Target	
Factual Accuracy	GPT-4o-mini judge vs ground truth	84.0%	≥ 80%	✓
Retrieval Precision @3	Correct filing in top-3 chunks?	100.0%	≥ 85%	✓
Hallucination Rate	GPT-4o-mini judge + cross-check	0.0%	≤ 10%	✓
LLM-as-Judge Score	GPT-4o-mini rates answer 1–5	4.76/5	≥ 4.0	✓

Overall we were pretty happy with these. Retrieval hitting 100% across all 25 questions means the vector search is consistently pulling from the right filing — that's the most important thing for a RAG system to get right. The 0% hallucination rate took the most iteration to achieve; we went through several versions of the system prompt before the model stopped adding details that weren't in the context.

Factual accuracy at 84% is solid. The main drag came from Microsoft — some of our MSFT questions asked for very specific dollar-amount changes in gross margin across fiscal years, and the model sometimes got the direction right but the exact figure slightly off. Apple was the cleanest (100%, 5.0 judge score), Alphabet was in the middle (88%, 4.50).

5. How the System Works

At query time, the user's question gets embedded with the same all-MiniLM-L6-v2 model we used during indexing. ChromaDB does a cosine similarity search and returns the top-K chunks (we default to 5). Users can also filter by company or filing type from the sidebar, which narrows the search before retrieval runs.

The retrieved chunks get assembled into a prompt with the user's question and sent to GPT-4o-mini. The system prompt design was Harshil's main contribution. The key constraints we built into it: only use the retrieved context, cite every claim with company/filing/period/section, copy numbers exactly as they appear in the filing, and explicitly say 'I don't have enough information' rather than guessing. Getting all four of those to hold together under edge cases took several rounds of testing and refinement.

The Streamlit UI (Bhuvan's work) has company and filing-type filters in the sidebar, a question input in the main area, and an expandable sources panel that shows the exact filing excerpts each part of the answer came from. There's also an evaluation tab that shows the metric scores once you've run the eval script.

6. Running the Project

```
pip install -r requirements.txt  
python fp_download.py # downloads the 12 EDGAR filings  
python fp_build_kb.py # parses, chunks, embeds, loads into ChromaDB  
python fp_evaluate.py # runs the 25-question eval and saves results  
streamlit run fp_app.py # launches the UI at localhost:8501
```

The download and build steps check what's already there before running, so it's safe to re-run them without creating duplicates. The Streamlit app also works before the knowledge base is built — it shows sample answers — and automatically switches to live RAG once ChromaDB is populated.